

Signal processing (`scipy.signal`)

Convolution

Convolution

B-splines

Filtering

Filter design

<code>convolve</code> (in1, in2[, mode, method])	Convolve two 1-dimensional arrays. Continuous-time linear systems
<code>correlate</code> (in1, in2[, mode, method])	Cross-correlate two N-dimensional arrays. Discrete-time linear systems LTI representations
<code>fftconvolve</code> (in1, in2[, mode, axes])	Convolve two N-dimensional arrays using FFT. Waveforms
<code>oaconvolve</code> (in1, in2[, mode, axes])	Convolve two N-dimensional arrays using the overlap-add method. Window functions Peak finding
<code>convolve2d</code> (in1, in2[, mode, boundary, fillvalue])	Convolve two 2-dimensional arrays. Spectral analysis Chirp Z-transform and Z
<code>correlate2d</code> (in1, in2[, mode, boundary, ...])	Cross-correlate two 2-dimensional arrays.
<code>sepfir2d</code> (input, hrow, hcol)	Convolve with a 2-D separable FIR filter.
<code>choose_conv_method</code> (in1, in2[, mode, measure])	Find the fastest convolution/correlation method.
<code>correlation_lags</code> (in1_len, in2_len[, mode])	Calculates the lag / displacement indices array for

B-splines

<code>gauss_spline</code> (x, n)	Gaussian approximation to B-spline basis function of order n.
<code>cspline1d</code> (signal[, lamb])	Compute cubic spline coefficients for rank-1 array.
<code>qspline1d</code> (signal[, lamb])	Compute quadratic spline coefficients for rank-1 array.
<code>cspline2d</code> (signal[, lamb, precision])	Coefficients for 2-D cubic (3rd order) B-spline.
<code>qspline2d</code> (signal[, lamb, precision])	Coefficients for 2-D quadratic (2nd order) B-spline.
<code>cspline1d_eval</code> (cj, newx[, dx, x0])	Evaluate a cubic spline at the new set of points.
<code>qspline1d_eval</code> (cj, newx[, dx, x0])	Evaluate a quadratic spline at the new set of points.
<code>spline_filter</code> (lin[, lmbda])	Smoothing spline (cubic) filtering of a rank-2 array.

Filtering

<code>order_filter</code> (a, domain, rank)	Perform an order filter on an N-D array.
---	--

<code>medfilt</code> (volume[, kernel_size])	Perform a median filter on an N-dimensional array.
<code>medfilt2d</code> (input[, kernel_size])	Median filter a 2-dimensional array.
<code>wiener</code> (im[, mysize, noise])	Perform a Wiener filter on an N-dimensional array.
<code>symiirorder1</code> (signal, c0, z1[, precision])	Implement a smoothing IIR filter with mirror-symmetric boundary conditions using a cascade of first-order sections.
<code>symiirorder2</code> (input, r, omega[, precision])	Implement a smoothing IIR filter with mirror-symmetric boundary conditions using a cascade of second-order sections.
<code>lfilter</code> (b, a, x[, axis, zi])	Filter data along one-dimension with an IIR or FIR filter.
<code>lfiltic</code> (b, a, y[, x])	Construct initial conditions for lfilter given input and output vectors.
<code>lfilter_zi</code> (b, a)	Construct initial conditions for lfilter for step response steady-state.
<code>filtfilt</code> (b, a, x[, axis, padtype, padlen, ...])	Apply a digital filter forward and backward to a signal.

<code>savgol_filter</code> (x, window_length, polyorder[, ...])	Apply a Savitzky-Golay filter to an array.
<code>deconvolve</code> (signal, divisor)	Deconvolves <code>divisor</code> out of <code>signal</code> using inverse filtering.
<code>sosfilt</code> (sos, x[, axis, zi])	Filter data along one dimension using cascaded second-order sections.
<code>sosfilt_zi</code> (sos)	Construct initial conditions for <code>sosfilt</code> for step response steady-state.
<code>sosfiltfilt</code> (sos, x[, axis, padtype, padlen])	A forward-backward digital filter using cascaded second-order sections.
<code>hilbert</code> (x[, N, axis])	FFT-based computation of the analytic signal.
<code>hilbert2</code> (x[, N, axes])	Compute the '2-D' analytic signal of x.
<code>envelope</code> (z[, bp_in, n_out, squared, ...])	Compute the envelope of a real- or complex-valued signal.
<code>decimate</code> (x, q[, n, ftype, axis, zero_phase])	Downsample the signal after applying an anti-aliasing filter.
<code>detrend</code> (data[, axis, type, bp, overwrite_data])	Remove linear or constant trend along axis from data.
<code>resample</code> (x, num[, t, axis, window, domain])	Resample x to <i>num</i> samples

	using the Fourier method along the given <i>axis</i> .
<code>resample_poly</code> (x, up, down[, axis, window, ...])	Resample x along the given axis using polyphase filtering.
<code>upfirdn</code> (h, x[, up, down, axis, mode, cval])	Upsample, FIR filter, and downsample.

Filter design

<code>bilinear</code> (b, a[, fs])	Calculate a digital IIR filter from an analog transfer function by utilizing the bilinear transform.
<code>bilinear_zpk</code> (z, p, k, fs)	Return a digital IIR filter from an analog one using a bilinear transform.
<code>findfreqs</code> (num, den, N[, kind])	Find array of frequencies for computing the response of an analog filter.
<code>firls</code> (numtaps, bands, desired, *[, weight, fs])	FIR filter design using least-squares error minimization.
<code>firwin</code> (numtaps, cutoff, *[, width, window, ...])	FIR filter design using the window method.
<code>firwin2</code> (numtaps, freq, gain, *[, nfreqs, ...])	FIR filter design using the window method.
<code>firwin_2d</code> (hsize, window, *[, fc, fs, ...])	2D FIR filter design using the

	window method.
<code>freqs</code> (b, a[, worN, plot])	Compute frequency response of analog filter.
<code>freqs_zpk</code> (z, p, k[, worN])	Compute frequency response of analog filter.
<code>freqz</code> (b[, a, worN, whole, plot, fs, ...])	Compute the frequency response of a digital filter.
<code>sosfreqz</code> (*args, **kwargs)	Compute the frequency response of a digital filter in SOS format (legacy).
<code>freqz_sos</code> (sos[, worN, whole, fs])	Compute the frequency response of a digital filter in SOS format.
<code>freqz_zpk</code> (z, p, k[, worN, whole, fs])	Compute the frequency response of a digital filter in ZPK form.
<code>gammatone</code> (freq, ftype[, order, numtaps, fs, ...])	Gammatone filter design.
<code>group_delay</code> (system[, w, whole, fs])	Compute the group delay of a digital filter.
<code>iirdesign</code> (wp, ws, gpass, gstop[, analog, ...])	Complete IIR digital and analog filter design.
<code>iirfilter</code> (N, Wn[, rp, rs, btype, analog, ...])	IIR digital and analog filter design given order and critical points.
<code>kaiser_atten</code> (numtaps, width)	Compute the attenuation of a Kaiser FIR filter.

<code>kaiser_beta</code> (<i>a</i>)	Compute the Kaiser parameter <i>beta</i> , given the attenuation <i>a</i> .
<code>kaiserord</code> (ripple, width)	Determine the filter window parameters for the Kaiser window method.
<code>minimum_phase</code> (<i>h</i> [, <i>method</i> , <i>n_fft</i> , <i>half</i>])	Convert a linear-phase FIR filter to minimum phase
<code>savgol_coeffs</code> (<i>window_length</i> , <i>polyorder</i> [, ...])	Compute the coefficients for a 1-D Savitzky-Golay FIR filter.
<code>remez</code> (<i>numtaps</i> , <i>bands</i> , <i>desired</i> , *[, <i>weight</i> , ...])	Calculate the minimax optimal filter using the Remez exchange algorithm.
<code>unique_roots</code> (<i>p</i> [, <i>tol</i> , <i>rtype</i>])	Determine unique roots and their multiplicities from a list of roots.
<code>residue</code> (<i>b</i> , <i>a</i> [, <i>tol</i> , <i>rtype</i>])	Compute partial-fraction expansion of $b(s) / a(s)$.
<code>residuez</code> (<i>b</i> , <i>a</i> [, <i>tol</i> , <i>rtype</i>])	Compute partial-fraction expansion of $b(z) / a(z)$.
<code>invres</code> (<i>r</i> , <i>p</i> , <i>k</i> [, <i>tol</i> , <i>rtype</i>])	Compute $b(s)$ and $a(s)$ from partial fraction expansion.
<code>invresz</code> (<i>r</i> , <i>p</i> , <i>k</i> [, <i>tol</i> , <i>rtype</i>])	Compute $b(z)$ and $a(z)$ from partial fraction expansion.
<code>BadCoefficients</code>	Warning about badly conditioned filter coefficients

Lower-level filter design functions:

<code>abcd_normalize</code> ([A, B, C, D])	Check state-space matrices compatibility and ensure they are 2d arrays.
<code>band_stop_obj</code> (wp, ind, passb, stopb, gpass, ...)	Band Stop Objective Function for order minimization.
<code>besselap</code> (N[, norm, xp, device])	Return (z,p,k) for analog prototype of an Nth-order Bessel filter.
<code>buttap</code> (N, *[, xp, device])	Return (z,p,k) for analog prototype of Nth-order Butterworth filter.
<code>cheb1ap</code> (N, rp, *[, xp, device])	Return (z,p,k) for Nth-order Chebyshev type I analog lowpass filter.
<code>cheb2ap</code> (N, rs, *[, xp, device])	Return (z,p,k) for Nth-order Chebyshev type II analog lowpass filter.
<code>ellipap</code> (N, rp, rs, *[, xp, device])	Return (z,p,k) of Nth-order elliptic analog lowpass filter.
<code>lp2bp</code> (b, a[, wo, bw])	Transform a lowpass filter prototype to a bandpass filter.
<code>lp2bp_zpk</code> (z, p, k[, wo, bw])	Transform a lowpass filter prototype to a bandpass filter.
<code>lp2bs</code> (b, a[, wo, bw])	Transform a lowpass filter prototype to a bandstop filter.
<code>lp2bs_zpk</code> (z, p, k[, wo, bw])	Transform a lowpass filter prototype to a bandstop filter.

<code>lp2hp</code> (b, a[, wo])	Transform a lowpass filter prototype to a highpass filter.
<code>lp2hp_zpk</code> (z, p, k[, wo])	Transform a lowpass filter prototype to a highpass filter.
<code>lp2lp</code> (b, a[, wo])	Transform a lowpass filter prototype to a different frequency.
<code>lp2lp_zpk</code> (z, p, k[, wo])	Transform a lowpass filter prototype to a different frequency.
<code>normalize</code> (b, a)	Normalize numerator/denominator of a continuous-time transfer function.

Matlab-style IIR filter design

<code>butter</code> (N, Wn[, btype, analog, output, fs])	Butterworth digital and analog filter design.
<code>buttord</code> (wp, ws, gpass, gstop[, analog, fs])	Butterworth filter order selection.
<code>cheby1</code> (N, rp, Wn[, btype, analog, output, fs])	Chebyshev type I digital and analog filter design.
<code>cheb1ord</code> (wp, ws, gpass, gstop[, analog, fs])	Chebyshev type I filter order selection.
<code>cheby2</code> (N, rs, Wn[, btype, analog, output, fs])	Chebyshev type II digital and

	analog filter design.
<code>cheb2ord</code> (wp, ws, gpass, gstop[, analog, fs])	Chebyshev type II filter order selection.
<code>ellip</code> (N, rp, rs, Wn[, btype, analog, output, fs])	Elliptic (Cauer) digital and analog filter design.
<code>ellipord</code> (wp, ws, gpass, gstop[, analog, fs])	Elliptic (Cauer) filter order selection.
<code>bessel</code> (N, Wn[, btype, analog, output, norm, fs])	Bessel/Thomson digital and analog filter design.
<code>iirnotch</code> (w0, Q[, fs, xp, device])	Design second-order IIR notch digital filter.
<code>iirpeak</code> (w0, Q[, fs, xp, device])	Design second-order IIR peak (resonant) digital filter.
<code>iircomb</code> (w0, Q[, ftype, fs, pass_zero, xp, ...])	Design IIR notching or peaking digital comb filter.

Continuous-time linear systems

<code>lti</code> (*system)	Continuous-time linear time invariant system base class.
<code>StateSpace</code> (*system, **kwargs)	Linear Time Invariant system in state-space form.
<code>TransferFunction</code> (*system, **kwargs)	Linear Time Invariant system class in transfer function form.
<code>ZerosPolesGain</code> (*system, **kwargs)	Linear Time Invariant system class in

	zeros, poles, gain form.
<code>lsim</code> (system, U, T[, X0, interp])	Simulate output of a continuous-time linear system.
<code>impulse</code> (system[, X0, T, N])	Impulse response of continuous-time system.
<code>step</code> (system[, X0, T, N])	Step response of continuous-time system.
<code>freqresp</code> (system[, w, n])	Calculate the frequency response of a continuous-time system.
<code>bode</code> (system[, w, n])	Calculate Bode magnitude and phase data of a continuous-time system.

Discrete-time linear systems

<code>dti</code> (*system, **kwargs)	Discrete-time linear time invariant system base class.
<code>StateSpace</code> (*system, **kwargs)	Linear Time Invariant system in state-space form.
<code>TransferFunction</code> (*system, **kwargs)	Linear Time Invariant system class in transfer function form.
<code>ZerosPolesGain</code> (*system, **kwargs)	Linear Time Invariant system class in zeros, poles, gain form.
<code>dlsim</code> (system, u[, t, x0])	Simulate output of a discrete-time linear system.
<code>dimpulse</code> (system[, x0, t, n])	Impulse response of discrete-time

	system.
<code>dstep</code> (system[, x0, t, n])	Step response of discrete-time system.
<code>dfreqresp</code> (system[, w, n, whole])	Calculate the frequency response of a discrete-time system.
<code>dbode</code> (system[, w, n])	Calculate Bode magnitude and phase data of a discrete-time system.

LTI representations

<code>tf2zpk</code> (b, a)	Return zero, pole, gain (z, p, k) representation from a numerator, denominator representation of a linear filter.
<code>tf2sos</code> (b, a[, pairing, analog])	Return second-order sections from transfer function representation
<code>tf2ss</code> (num, den)	Transfer function to state-space representation.
<code>zpk2tf</code> (z, p, k)	Return polynomial transfer function representation from zeros and poles
<code>zpk2sos</code> (z, p, k[, pairing, analog])	Return second-order sections from zeros, poles, and gain of a system
<code>zpk2ss</code> (z, p, k)	Zero-pole-gain

	representation to state-space representation
<code>ss2tf</code> (A, B, C, D[, input])	State-space to transfer function.
<code>ss2zpk</code> (A, B, C, D[, input])	State-space representation to zero-pole-gain representation.
<code>sos2zpk</code> (sos)	Return zeros, poles, and gain of a series of second-order sections
<code>sos2tf</code> (sos)	Return a single transfer function from a series of second-order sections
<code>cont2discrete</code> (system, dt[, method, alpha])	Transform a continuous to a discrete state-space system.
<code>place_poles</code> (A, B, poles[, method, rtol, maxiter])	Compute K such that eigenvalues (A - dot(B, K))=poles.

Waveforms

<code>chirp</code> (t, f0, t1, f1[, method, phi, ...])	Frequency-swept cosine generator.
<code>gausspulse</code> (t[, fc, bw, bwr, tpr, retquad, ...])	Return a Gaussian modulated sinusoid:
<code>max_len_seq</code> (nbits[, state, length, taps])	Maximum length sequence (MLS)

	generator.
<code>sawtooth</code> (t[, width])	Return a periodic sawtooth or triangle waveform.
<code>square</code> (t[, duty])	Return a periodic square-wave waveform.
<code>sweep_poly</code> (t, poly[, phi])	Frequency-swept cosine generator, with a time-dependent frequency.
<code>unit_impulse</code> (shape[, idx, dtype])	Unit impulse signal (discrete delta function) or unit basis vector.

Window functions

For window functions, see the `scipy.signal.windows` namespace.

In the `scipy.signal` namespace, there is a convenience function to obtain these windows by name:

<code>get_window</code> (window, Nx[, fftbins, xp, device])	Convenience function for creating various windows.
---	--

Peak finding

<code>argrelemin</code> (data[, axis, order, mode])	Calculate the relative minima of <i>data</i> .
<code>argrelmax</code> (data[, axis, order, mode])	Calculate the relative maxima of <i>data</i> .

<code>argrextrema</code> (data, comparator[, axis, ...])	Calculate the relative extrema of <i>data</i> .
<code>find_peaks</code> (x[, height, threshold, distance, ...])	Find peaks inside a signal based on peak properties.
<code>find_peaks_cwt</code> (vector, widths[, wavelet, ...])	Find peaks in a 1-D array with wavelet transformation.
<code>peak_prominences</code> (x, peaks[, wlen])	Calculate the prominence of each peak in a signal.
<code>peak_widths</code> (x, peaks[, rel_height, ...])	Calculate the width of each peak in a signal.

Spectral analysis

<code>periodogram</code> (x[, fs, window, nfft, detrend, ...])	Estimate power spectral density using a periodogram.
<code>welch</code> (x[, fs, window, nperseg, noverlap, ...])	Estimate power spectral density using Welch's method.
<code>csd</code> (x, y[, fs, window, nperseg, noverlap, ...])	Estimate the cross power spectral density, P_{xy} , using Welch's method.
<code>coherence</code> (x, y[, fs, window, nperseg, ...])	Estimate the magnitude squared coherence estimate, C_{xy} , of discrete-time signals X and Y using Welch's method.
<code>spectrogram</code> (x[, fs, window, nperseg, ...])	Compute a spectrogram with consecutive Fourier transforms

	(legacy function).
<code>lombscargle</code> (x, y, freqs, *[, precenter, ...])	Compute the generalized Lomb-Scargle periodogram.
<code>vectorstrength</code> (events, period)	Determine the vector strength of the events corresponding to the given period.
<code>ShortTimeFFT</code> (win, hop, fs, *[, fft_mode, ...])	Provide a parametrized discrete Short-time Fourier transform (stft) and its inverse (istft).
<code>closest_STFT_dual_window</code> (win, hop[, ...])	Calculate the STFT dual window of a given window closest to a desired dual window.
<code>stft</code> (x[, fs, window, nperseg, noverlap, ...])	Compute the Short Time Fourier Transform (legacy function).
<code>istft</code> (Zxx[, fs, window, nperseg, noverlap, ...])	Perform the inverse Short Time Fourier transform (legacy function).
<code>check_COLA</code> (window, nperseg, noverlap[, tol])	Check whether the Constant OverLap Add (COLA) constraint is met (legacy function).
<code>check_NOLA</code> (window, nperseg, noverlap[, tol])	Check whether the Nonzero Overlap Add (NOLA) constraint is met.

Chirp Z-transform and Zoom FFT

<code>czt</code> (<code>x</code> [, <code>m</code> , <code>w</code> , <code>a</code> , <code>axis</code>])	Compute the frequency response around a spiral in the Z plane.
<code>zoom_fft</code> (<code>x</code> , <code>fn</code> [, <code>m</code> , <code>fs</code> , <code>endpoint</code> , <code>axis</code>])	Compute the DFT of <code>x</code> only for frequencies in range <code>fn</code> .
<code>CZT</code> (<code>n</code> [, <code>m</code> , <code>w</code> , <code>a</code>])	Create a callable chirp z-transform function.
<code>ZoomFFT</code> (<code>n</code> , <code>fn</code> [, <code>m</code> , <code>fs</code> , <code>endpoint</code>])	Create a callable zoom FFT transform function.
<code>czt_points</code> (<code>m</code> [, <code>w</code> , <code>a</code>])	Return the points at which the chirp z-transform is computed.

The functions are simpler to use than the classes, but are less efficient when using the same transform on many arrays of the same length, since they repeatedly generate the same chirp signal with every call. In these cases, use the classes to create a reusable function instead.

© Copyright 2008, The SciPy community.

Built with the [PyData Sphinx Theme](#) 0.16.1.

Created using [Sphinx](#) 8.1.3.